Multimedia Tools and Applications (2025) 84:39143–39169
https://doi.org/10.1007/s11042-025-20602-x

1250: HERITAGE PRESERVATION IN THE DIGITAL AGE: ADVANCES IN MACHINE LEARNING, MONOMODAL AND MULTIMODAL PROCESSING, AND HUMAN-MACHINE INTERACTION

Text line segmentation approach combining deep learning model and traditional image processing techniques - application to transliteration of Cham manuscripts

Tien Nam Nguyen[1] · Jean Christophe Burie[1] · Thi Lan Le[2] · Anne-Valerie Schweyer[3]

Received: 17 April 2024 / Revised: 14 November 2024 / Accepted: 30 December 2024 /
Published online: 21 January 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Text line segmentation in historical document images is a crucial task in document analysis and understanding. Historical documents often feature degraded and complex layouts, and they may exhibit distinct writing styles and languages, posing challenges for text line segmentation. Our study focuses on the text line segmentation task for Cham manuscript images. Cham manuscripts, written on Chinese paper and composed in Middle Cham, serve as precious sources to understand the daily lives of the inhabitants of the Champa Kingdom and provide insights into Champa possessions and their management. To address the challenges of text line segmentation in Cham manuscript images, this study proposes an approach that combines deep learning methods with traditional image processing techniques. Our approach takes into account the characteristics of Cham manuscripts to effectively tackle the challenges posed by these documents. Specifically, the procedure begins by leveraging pre-trained deep learning models and includes a fine-tuning step to extract the candidate baseline. Then, various image post-processing algorithms are introduced to further improve segmentation accuracy. Extensive experiments on our Cham manuscripts dataset consisting of 627 manuscripts with approximately 8300 segmented text lines using different evaluation criteria demonstrate the significant superiority of our approach over existing methods. Our method outperforms state-of-the-art performance by a considerable margin.

✉ Tien Nam Nguyen
tnguye28@univ-lr.fr

Jean Christophe Burie
jcburie@univ-lr.fr

Thi Lan Le
lan.lethi1@hust.edu.vn

Anne-Valerie Schweyer
anne-valerie.schweyer@cnrs.fr

1 Laboratoire Informatique, Image et Interaction (L3i), La Rochelle University, La Rochelle 17000, France

2 School of Electrical and Electronic Engineering (SEEE), Hanoi University of Science and Technology, Hanoi 10000, Vietnam

3 Institut Français de Recherches sur l'Asie du l'Est (IFRAE), CNRS, Paris, France

Keywords Document image analysis · Text line segmentation · Cham manuscripts · Transliteration

1 Introduction

Historical documents contain many invaluable information which reflects the events of the past. Each document was produced using different material, writing format, and language leading to different methods for exploring these documents. Document analysis has been studied for long periods and has achieved promising results on printed documents.

However, when it comes to handwritten documents like Balinese manuscripts [1] or Khmer manuscripts [2], as well as Cham manuscripts [3], which exhibit distinct writing styles and languages, the overall performance of existing methods drops significantly [1]. Therefore, further adaptation of the baseline approaches is usually necessary to achieve satisfactory results in these cases.

Analyzing a document generally involves two main steps: analysis and recognition. During analysis, the structure of the document, including pages, paragraphs, text lines, and words, is identified [4]. Recognition poses a challenge to researchers because the quality of preceding steps influences the accuracy of this step, particularly the segmentation of text lines. In the case of the Cham language studied in this work, we have a transliteration task (converting text from one script to another according to specific rules) which depends heavily on the accurate recognition of descenders, ascenders, and diacritics. In general, transliteration can be categorized into two main types: text-to-text, which converts text directly from one script to another, and image-to-text, which is similar to Optical Character Recognition (OCR) systems. For our task, the objective aligns with image-to-text transliteration, where we aim to convert images of text into a different script. Therefore, accurate analysis of text-lines is essential in the whole process.

This study focuses on text line analysis of Cham manuscripts. Since the demise of Champa kingdoms during the 19th century, the Cham language that originated and developed from 2nd century, is no longer used among the descendants of the Champa. The lack of transmission of knowledge and documents of the Cham culture makes the study of this language difficult for epigraphists and historians.

Considering the Cham language and the history of the Champa kingdom (which spanned from the 2nd to the 19th centuries and was located in the central region of present-day Vietnam), historical documents can be classified into two main types: inscriptions and manuscripts. The inscriptions, mainly in the Old Cham (or Sanskrit) language, are engraved on stone surfaces. They mainly document religious gifts, and praises to deities, kings, and their families. Cham inscription images are obtained through a stamping and scanning process. On the other hand, the manuscripts, written on "Chinese" paper, are composed in Middle Cham and have also been digitized. These documents shed light on the daily lives of the inhabitants of the Champa Kingdom and provide insight into the Champa possessions and their management.

In previous work [5], we studied the segmentation of text-lines for Cham inscriptions. However, as the complexity of lines of text increases (manifested by skewed text, variable layouts, short lines, or abundant diacritics), the effectiveness of approaches based on histogram projection profile decreases on Cham manuscripts. Such methods struggle to accurately detect certain features, leading to errors such as missing short text lines, incorrect line lengths, and inaccurate identification of candidate text-lines as shown in Fig. 1.

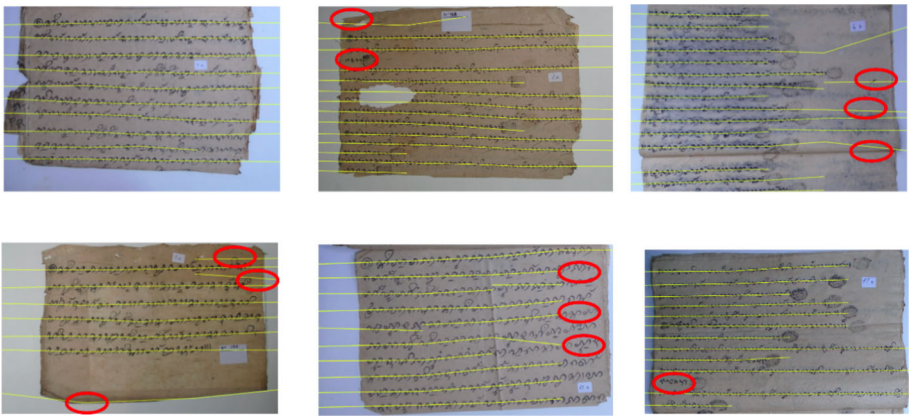


Fig. 1 Text line segmentation results by histogram projection profile on Cham manuscripts. Missing short text lines, incorrect line lengths, and inaccurate identification of candidate text lines are marked by red color

It is imperative to explore alternative approaches to address this challenge. Currently, deep learning-based methods have been applied to various tasks and have shown significant improvements compared to traditional image processing techniques. Furthermore, considering that the Cham inscription dataset used in previous work is relatively small, it is crucial to assess the robustness of the methods on a larger dataset. In this study, we focus on a vast collection of Cham manuscripts, characterized by diverse writing layouts, as shown in Fig. 2. The differences in this collection compared to Cham inscriptions are as follows:

- Written materials: Manuscripts are on Chinese paper instead of inscribed on stone surfaces.
- Writing style: In-homogeneous writing, including both straight and skewed text lines within the same document, as well as disconnected text lines.

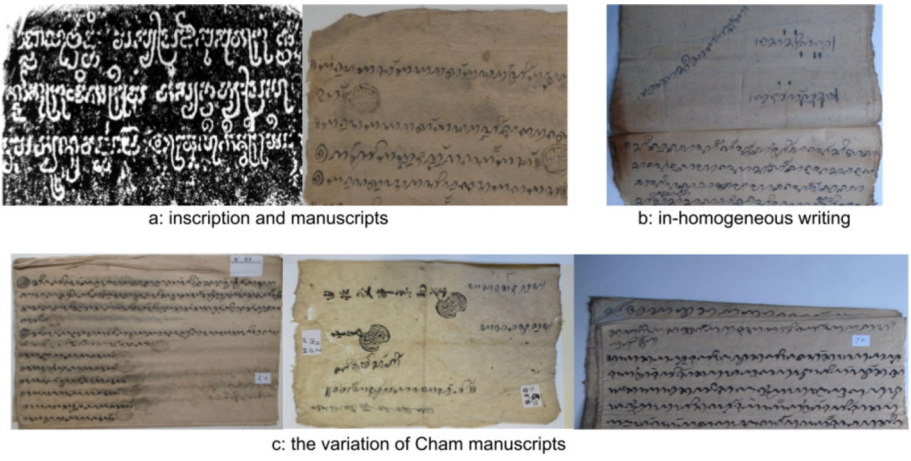


Fig. 2 (a): difference between inscription and manuscript (b, c): examples of manuscripts with various characteristics

- Volume: The collection comprises a large number of images (up to 600 images approximately 8300 text-lines) with various types of noise, such as ink bleed, page corruption, ink composition variations, and mixing of languages.

These differences require the development of a new approach to effectively address them. Currently, the predominant approach to this task relies on deep learning models for basic detection, followed by post-processing methods to refine the results. Numerous pre-trained deep learning models, trained on various datasets [6, 7], or commercial software solutions such as [8, 9], have demonstrated their effectiveness in basic segmentation. However, when dealing with new documents with distinct characteristics, post-processing steps must be adapted based on the document specific layout assumptions. For example, in the context of Cham inscriptions [3], an additional cost function based on the writing style of the documents has been introduced. However, the effectiveness of these results remains constrained by challenges in baseline extraction and the incorporation of the additional cost function. This paper extends the work reported in [5] with three main contributions:

- Firstly, based on the observation of limitations in baseline extraction in the proposed method for Cham inscriptions, we introduce new approaches for Cham manuscript documents using deep learning models. These methods are deployed and thoroughly evaluated on a large Cham manuscript collection.
- Secondly, we modify the original seam carving algorithm to accommodate different lengths of text-line.
- Thirdly, we propose the different post-processing algorithms to further refine the results obtained from the deep learning model. Experimental results show that these additional steps contribute to improving overall performance.
- Finally,
  for a comprehensive evaluation, instead of using typical segmentation metrics, we evaluate the segmentation results through the transliteration step using a given recognition model. The accuracy of the proposed method is confirmed using both criteria and off-the-shelf software.

The remainder of this paper is organized as follows: Section 2 provides a brief review of the literature on text line segmentation. Section 3 presents the proposed method. Summary of Cham manuscripts is presented in the Section 4. In Section 5, we detail and analyze experimental results. Finally, Section 6 concludes with a summary of our results and describes directions for future research.

2 Related works

A text-line refers to a group of characters, symbols, and words that are contiguous and spatially proximate, allowing for a straight line to be drawn through them [4]. Text-line segmentation methods can generally be categorized into three main approaches: top-down, bottom-up, and deep learning-based. For a comprehensive survey of this task, particularly concerning historical documents, interested readers can refer to [10].

Top-down approaches The top-down approach, pioneered by works such as the projection profiles method [11], relies on analyzing projection profiles that sum the number of pixels projected horizontally or vertically (depending on the direction under study). Peaks in the profile represent text segments (text lines), while valleys indicate spaces between lines (and

vice versa if the text is black and the background is white). To mitigate local maxima, the projection profile is often smoothed using techniques like a Gaussian filter [12] or a moving average filter [13]. To address skewed and curved lines, partial projection profiles are employed [14–17]. In this method, the document is vertically divided into sub-columns (or rows, depending on the layout). Each sub-column's local maxima in the projection profile are considered candidate lines. These candidates are then matched using predefined rules [17] or by measuring distances between them [14]. However, challenges arise when dealing with documents containing numerous lines of varying lengths or spacing between them. The projection profile values may not accurately identify candidate lines in such cases. In [18], based on the assumption of writing in the filled document (writing from left side to right), the partial projection profile (of first 100 to 200 columns) is used first to estimate the number of text lines and its candidate points. Text-line is created by computing the histogram in three directions: straight, upward, or downward.

Another top-down approach is the seam carving-based method [14, 19–21]. This technique uses seam carving to segment text-lines, aiming to minimize the accumulated energy map computed from the image. High energy values indicate text areas, while low energy values represent background areas. In [19], a two-step seam carving process is proposed. First, seam seeds, mainly representing the base text line, are generated from the seam-map generation in two passes. Then, separating lines are extracted by determining the lower and upper boundaries for each seam seed. Similarly, in [14] they proposed a two-step strategy using partial projection profiles. In [21], the Labeling, Cutting, Grouping (LCG) method is introduced, which relies on a new energy map and seam casting for text line segmentation. A related method is the A* path planning approach [22], which finds separation lines by minimizing different cost functions to traverse from the **s**tarting state (s) to the **g**oal state (g).

Bottom-up approaches In the case of bottom-up approaches, various techniques have been explored. For instance, in [23–26] different topological features such as blocks, connected components, search trees, and heuristic rules are used to extract text lines. One of the earliest methods, known as smearing, has been extensively employed in the literature [27–30]. The pioneering work in this domain was presented in [27], where a sequence of pixels is smeared horizontally (or vertically, depending on the document structure) over a defined distance. The connected components identified on the smeared image serve as the boundaries of text-lines. Traditional smearing methods typically operate on binary input documents. In [30], the authors introduce an Adaptive Local Contrast Maximization (ALCM) technique, which transforms the grayscale input image into a new map space where higher values correspond to text pixels. To address challenges posed by variable documents (such as shorter lines or irregular line spacing), the ALCM method is extended by incorporating different filters, such as the steerable filter [29] and the isotropic Gaussian filter [24]. In [24], instead of relying on a single filter, the smearing image L is generated through a combination of smoothed versions of the input image. Subsequently, a component tree is used for the binarization of the smearing image. Validated connected components are then identified as text-lines. To address issues such as labeling disconnected components or removing spurious lines, various functions are applied in a post-processing step. The final text-lines are derived by minimizing a defined objective function. In [26], based on the heuristics rules of the font size of printed documents, the connected components are aligned to create a text line. This method proves quite good results on some specific font types, but the robustness is not guaranteed. In general, a drawback of this approach arises when the structure of the text-line is fragmented, as the smearing process may fail to merge the components seamlessly. Additionally, if the space

between two consecutive lines is minimal (resulting in ascending and descending characters touching the upper/lower lines), the effectiveness of this method diminishes.

Deep learning based approaches Recently, there has been a sharp increase in the development of deep learning-based methods for text line segmentation [6, 31–37]. Drawing inspiration from the semantic segmentation and objection detection tasks in computer vision, various kind convolutional neural networks (CNNs) have been explored for addressing the text line segmentation problem [6, 31, 35, 36].

For the semantic segmentation, in [31], the authors adapted the fully convolutional networks (FCNs) [38] to address text line segmentation, leveraging an encoder-decoder architecture. Similarly, [36] used FCNs but with notable distinctions from [31]: there is no resolution change in the input image throughout the encoder and decoder, and dilated convolutions are employed instead of transpose convolutions or upsampling. Dilated convolutions are favored because detecting lines of text does not require extensive context for accurate segmentation. In [34], the original Unet architecture [39] was adapted for text line segmentation.

Considering the objection detection, in [37], the authors proposed a new model by modification the original faster R-CNN by using the Resnet50 and different scaling factors of the anchor boxes. The proposed architecture has a problem when the text line is skewed, the bounding box may touch both the previous and the next text-line.

Typically, results from deep learning models manifest themselves as disconnected or noisy components, where a single line is fragmented into multiple components. To refine the segmentation, post-processing steps incorporating heuristic cost functions [31] or line adjacency graphs [35] are used. In [33], a similar approach of integrating deep learning with image processing was adopted. Initially, a deep convolution network named ARU-Net is employed to classify each pixel into distinct classes such as baseline, separator, and others. Subsequently, superpixels are clustered and merged into text lines based on certain ad hoc information. In contrast, [32] adopted an unsupervised approach by training a Siamese network to discern the similarity between pairs of patches extracted randomly from a list of cropped patches of the original image. The trained model serves as an embedding network to extract image blobs, which are then segmented into appropriate text lines using energy minimization combining various cost functions.

Off-the-shelf system for document analysis As the scientific community delves deeper into the field of historical document analysis, advancements continue to arise through the adaptation of various techniques from disparate fields. To ensure widespread accessibility, certain software solutions have been developed and structured in a unified manner. These platforms can be readily deployed to end users without specialized technical expertise. Among the notable software solutions in this domain are Transkribus [8] and eScriptorium [9], LayoutParser [40].

Transkribus [8] is one of the first interactive computer-assisted platforms designed for transcribing, recognizing, and retrieving digitized historical documents. This versatile tool offers support for two key tasks: layout analysis and text recognition. Notably, Transkribus boasts an online server for facilitating the training process, effectively mitigating issues related to accessing training resources. Based on the work of Transkribus, eScriptorium [9] was developed, drawing inspiration from [41] for document layout analysis and leveraging Kraken[1] for recognition tasks. A notable enhancement over Transkribus is its user-definable architecture, allowing users to define their own models instead of relying solely on pre-existing

[1] https://kraken.re/main/index.html

ones, thereby augmenting its versatility and power. Additionally, eScriptorium provides a more user-friendly interface, featuring easier interactions and intuitive functionalities such as region visualization, bounding box adjustment, ground truth modification, and model import/export capabilities.

Recently, inspired by NLP foundation models like BERT [42], similar models have been increasingly applied to document processing tasks, leveraging their pre-trained capabilities to enhance information extraction and layout recognition. Layout Parser [40], for example, uses large-scale training data to tackle layout detection, table extraction, and character recognition, bringing substantial progress to automating complex document analysis. To foster collaboration and knowledge sharing, Layout Parser introduced a model hub, allowing users to upload and share custom-trained models that integrate seamlessly into existing frameworks, enhancing accessibility and adaptability across various document processing tasks. However, Layout Parser requires an axis-aligned bounding box, which can create excessively large boxes when handling skewed text lines, sometimes overlapping with adjacent lines. Similarly, fine-tuned models, like doc-UFCN, encounter a difficulty when non-overlapping bounding boxes are needed. This challenge was one reason for choosing a segmentation approach over object detection, as it simplifies ground-truth preparation. Figure 3 illustrates segmentation results on Cham manuscripts. We observe that the segmentation accuracy is high for shorter or relatively straight lines, but performance decreases significantly for longer, more skewed text lines.

All the tools offer the ability for users to both share and utilize numerous pre-trained models across various languages. This feature holds significant value in garnering attention from the historical document community, particularly for researchers in the humanities with limited technical expertise in artificial intelligence. In general, trained models or off-the-shelf systems often require adjustments to manage variations such as lines without ascenders or descenders, or cases where lines are interconnected, necessitating post-processing for further refinement. Additionally, baseline configurations may vary by document type, making additional fine-tuning essential to adapt models to new datasets and ensure accurate text interpretation.

3 An advanced approach for text line segmentation

In this study, we focus on Cham manuscripts, which are characterized by complex scripts featuring ascenders, descenders, and diacritics critical for accurate character transliteration. These complex script elements require careful consideration during text-line segmentation.

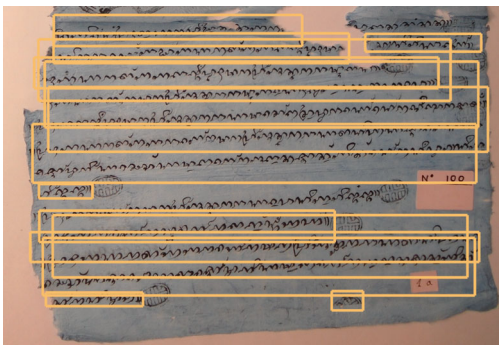


Fig. 3 Segmentation results from fine-tuning model of LayoutParser

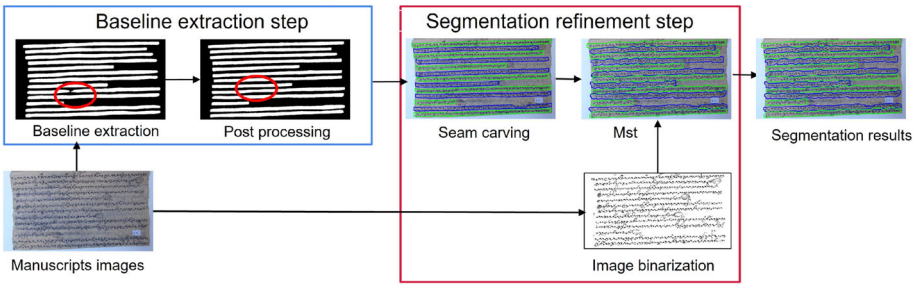


Fig. 4 Proposed pipeline for text line segmentation on Cham manuscripts

Notably, the layout of these manuscripts closely resembles well-studied datasets such as READ-BAD [42], DIVA-HisDB [43], and the Balinese palm leaf manuscript dataset [1]. This similarity allows us to leverage insights from these previous studies, which can significantly improve our results.

Our proposed method consists of two main steps: baseline extraction step and segmentation refinement step with seam carving and minimum spanning tree (MST). In detail, the first step involves using a fine-tuned docExtractor model to identify and extract text line baselines from the input manuscripts (blue block). Following baseline extraction, two post-processing algorithms (merging and splitting) further refine the extracted baselines. A customized seam carving method is then employed to enhance text line separation. Finally, additional refinement is made to the text lines generated through MST to ensure precise boundary delineation improving the accuracy of line separation. A pre-trained SAE [44] model is used to generate a binarized image of the manuscript, which serves as input for the minimum spanning tree (MST) algorithm. The proposed pipeline is illustrated in Fig. 4.

3.1 Baseline extraction step

For baseline extraction, two approaches can be considered: segmentation and object detection. We chose segmentation for its precision in localization and robustness to variations in text lines. In contrast, object detection may struggle to accurately capture the intricate contours

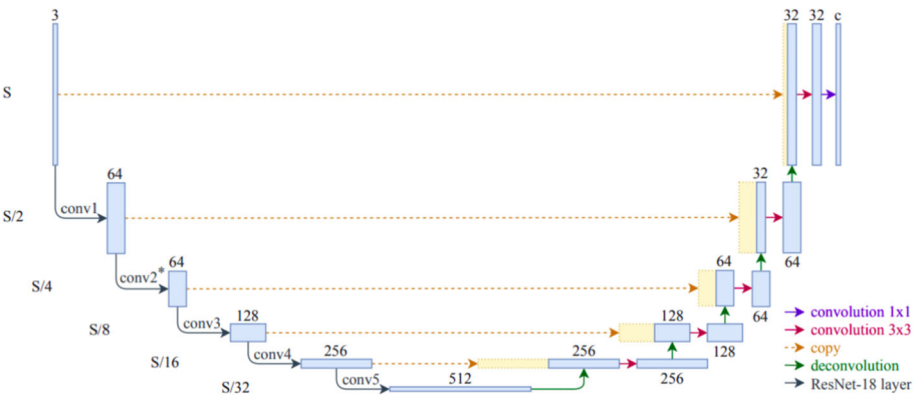


Fig. 5 docExtractor network architecture [7]

of curved text lines. Numerous pre-trained models have been rigorously evaluated across various datasets. Among these, docExtractor [7] stands out for its proven robustness in cross-dataset scenarios. Hence, we opted to begin with this pre-trained model (trained on IlluHisDoc dataset). docExtractor uses a U-Net architecture with an encoder-decoder structure based on a ResNet-18 backbone. This approach is similar to semantic segmentation, where the model classifies and identifies distinct document regions, such as text, decorations, captions, glyphs and tables. In the encoding phase, input images are progressively down-sampled through convolutional layers to extract high-level features. During decoding, the target image is reconstructed step-by-step using these learned features. Skip connections between encoding and decoding layers preserve spatial information, enhancing the accuracy and detail of the reconstructed image. An illustration of the docExtractor architecture is provided in Fig. 5. In Fig. 6, we present the results of baseline extraction using docExtractor on our Cham

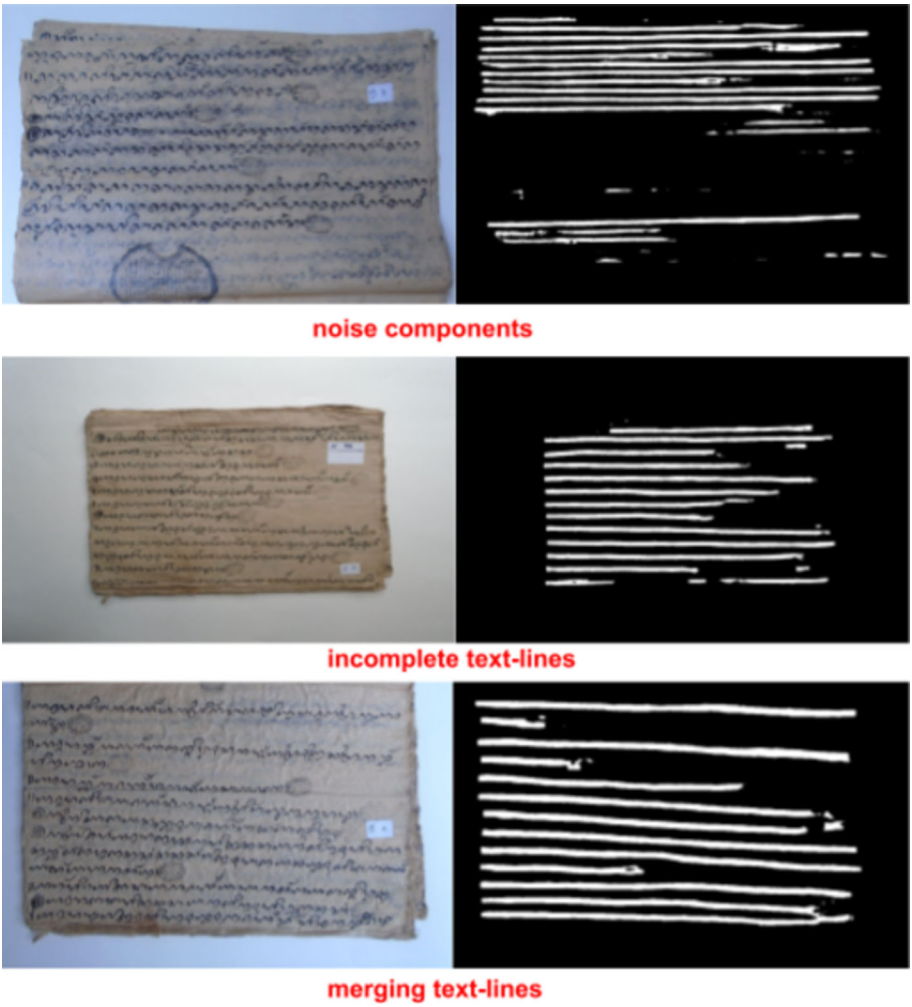


Fig. 6 Baseline results obtained using the pre-trained docExtractor on Cham manuscripts. (Left) Original image. (Right) Baseline results from docExtractor

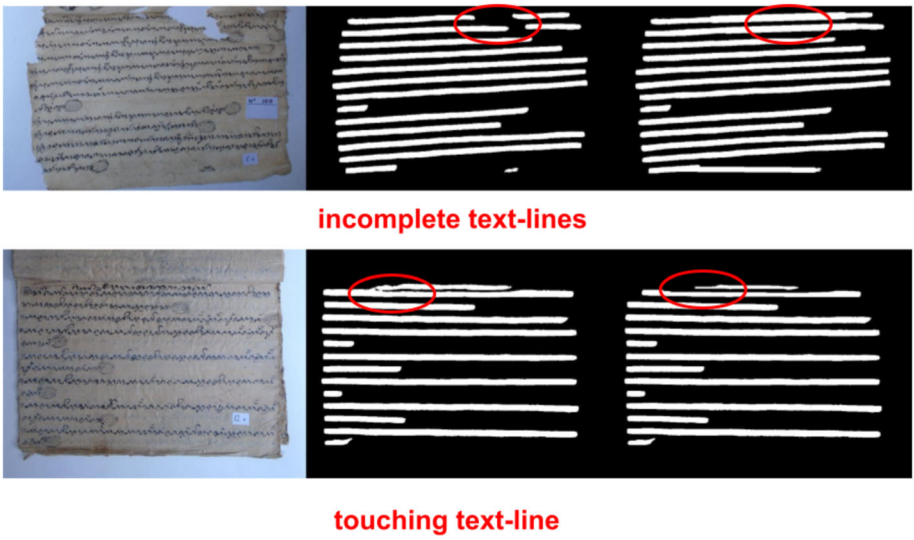


Fig. 7 Example of merging/disconnected of two consecutive text-lines. (Left) Original image, (middle) The red circle shows the connected components in contact. (Right) Result after the splitting/merging process

manuscripts. It is evident that when used as is, the model still exhibits errors such as noise components and incomplete text lines. After fine-tuning (see Fig. 7), many of these errors are significantly reduced. However, there are still cases where text lines appear disconnected (due to the page being torn) or touching (two text lines close to each other), presenting challenges for the model. In such cases, post-processing steps are required for proper refinement.

3.1.1 Post-processing results of baseline extraction

As noted earlier, the fine-tuned baseline extraction model still produces unsatisfactory results, including disconnected and touching text-lines. These errors lead to incorrect candidate text lines, necessitating adjustments to rectify these connected components. Consequently, we propose two algorithms for merging and splitting these components.

Merging text-lines

Inaccurate baseline extraction can cause a single line of text to be split into multiple connected components. Therefore, it becomes essential to reconnect disparate parts of the same line of text. After analyzing these problems, two components can be considered part of a single line of text if the average of their height coordinates is less than the T_{cch} threshold and the angle they form is less than T_{cca} threshold. Before merging, a skeletonization transformation is applied to facilitate the operation. The algorithm for checking these conditions is described in Algorithm 1.

The lines 7 and 8 of the Alg. 1, calculate the angle $ANGLE(cc^1)$ and $ANGLE(cc^2)$, we first fit a 1D linear model (a straight line) to all points within the connected component. Then, we compute the angle of this fitted line relative to the horizontal axis to represent the overall orientation of the cluster.

Algorithm 1 Condition checking algorithm for merging connected component.

Require: Two connected components cc^1, cc^2 of skeleton images; Thresholds T_{cch} for height and T_{cca} for angle.

$\overline{h_{cc1}} = AVERAGE(cc_h^1)$ ▷ cc_h^1 is height coordinate of all points in the cc^1
$\overline{h_{cc2}} = AVERAGE(cc_h^2)$ ▷ cc_h^2 is height coordinate of all points in the cc^2
$d \leftarrow ABS(\overline{h_{cc1}}, \overline{h_{cc2}})$
if $d \leq T_{cch}$ **then** ▷ Checking the distance between two connected components
    continue
else
    $\alpha_{cc1} \leftarrow ANGLE(cc^1)$ ▷ $ANGLE(cc^1)$ is the angle created by all points in connected component cc^1
    $\alpha_{cc2} \leftarrow ANGLE(cc^2)$ ▷ $ANGLE(cc^2)$ is the angle created by all points in connected component cc^2
    $d_{angle} \leftarrow ABS(\alpha_{cc1}, \alpha_{cc2})$
    if $d_{angle} \leq T_{cca}$ **then**
        Merging
    else
        $f \leftarrow POLYFIT(x, y)$ ▷ create a linear line from connected components. (x, y) is the coordinate of all points of $cc^1 \cup cc^2$, POLYFIT is a linear function (polynomial fitting) that approximate all points in the connected components cc^i. It is calculated by minimizing the least squares polynomial errors.
        $\hat{y} \leftarrow f(x)$
        $err_d \leftarrow MSE(\hat{y}, y)$ ▷ MSE is the mean square error to measure the linearity of text-line
        if $err_d \leq T_{err}$ **then**
            Continue
        else
            Merging
        end if
    end if
end if

Splitting text-lines The details of the conditions to split these components are given below in Algorithm 2. These connected components are considered candidates for splitting if their height exceeds the average height of all connected components and the errors in fitting all the points with a 1D linear function surpass a predefined threshold. The specifics of the conditions for splitting these components are outlined in Algorithm 2.

Algorithm 2 Splitting of the big connected components.

Require: List of all connected components cc^1, cc^2, ..., cc^L; Threshold T_{height}

if $cc_h^i \leq T_{height}$ **then**
    continue
else
    $f \leftarrow POLYFIT(x, y)$ ▷ create a linear line from connected components. x, y is the coordinate of all points in the connected components cc^i, POLYFIT is defined in algorithm 1
    $\hat{y} \leftarrow f(x)$
    $err_d \leftarrow MSE(\hat{y}, y)$ ▷ MSE is the mean square error to measure the linearity of text-line
    if $err_d \geq T_{err}$ **then**
        SPLIT cc^i
    end if
end if

Now that we have identified suitable candidate baselines for the manuscripts, the next step is to assign them to each text line, encompassing all its descenders and ascenders. We use the seam carving algorithm, as described in [5], without using any additional cost function.

3.2 Segmentation refinement step

3.2.1 Seam carving

As shown in Fig. 8, the segmentation results from the baseline model fail to accurately cover the text lines, with initial segmentation often misaligned with the text regions. To address these limitations, seam carving is applied to dynamically refine the boundaries of the segmented areas by adjusting pixel content. Subsequently, a Minimum Spanning Tree (MST) algorithm is used to further refine and optimize the segmentation results. The combination of seam carving with MST ensures more precise delineation of the text-lines, significantly enhancing the overall performance of the baseline extraction model.

Energy map construction To cast seams, an energy map must be constructed to guide the seams on how to separate the text lines. Several methods can be used to compute the energy map including Gray-level Distance Transform (GDT) [45], gradient map (GM) [14], or a combination of background energy and text energy map (LCG) as proposed in [21].

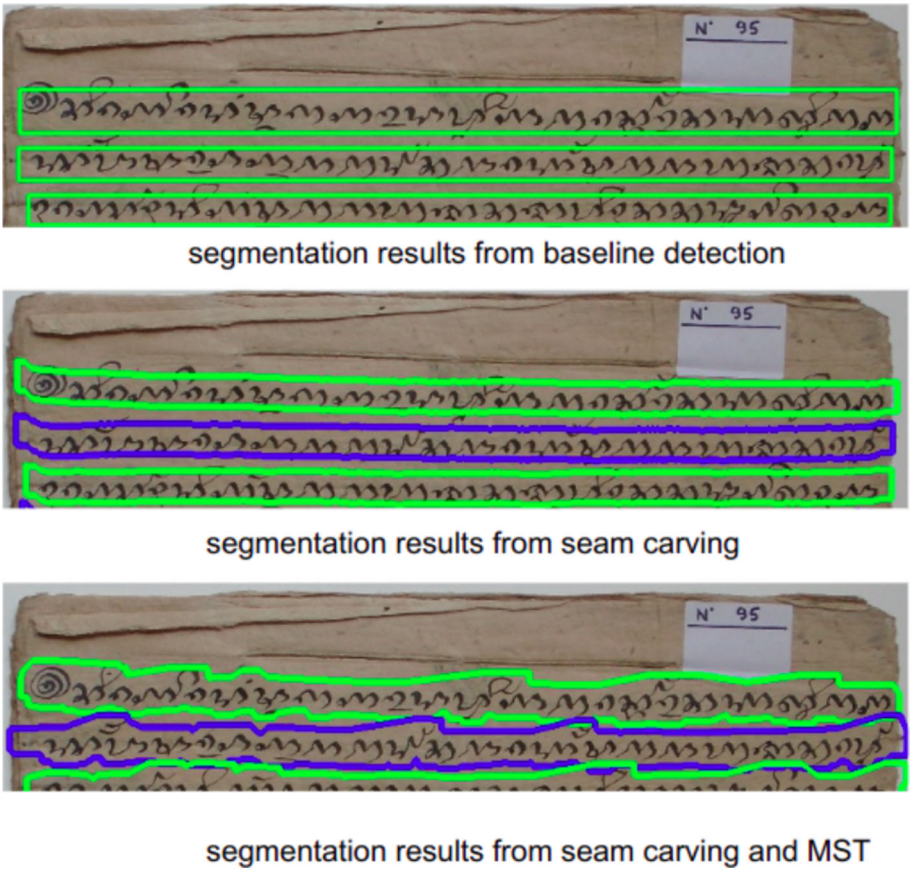


Fig. 8 Segmentation results before and after the implementation of seam carving and Minimum Spanning Tree (MST) algorithms, arranged sequentially from top to bottom

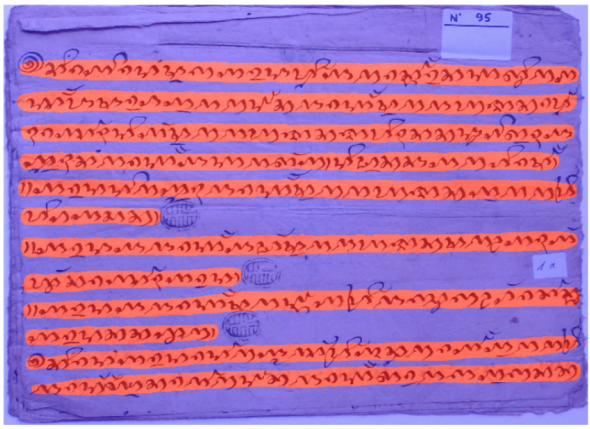


Fig. 9 Heat map of energy map computed from baseline extraction image. The red color (resp. purple) represents high energy (resp. low)

Depending on the workflow proposed in these methods, each energy map function offers its own advantages. For example, in the context of the Cham inscriptions [5], the LCG method [21] was used. However, in this study, we discovered that the image extracted from baseline extraction is sufficient to guide the separation of text-lines. Although we experimented with the use of a dedicated cost function as in [5], it did not result in a significant improvement overall. Therefore, we opted to solely use the baseline extraction image as the energy map for casting seams. Figure 9 illustrates the energy map obtained from the baseline extraction image.

Cast seams In this step, we generate separate seams for the text-line based on the energy map and the position of the candidate text-line. Instead of using the same position for the entire line, which can result in incorrect boundaries due to variations in position and length, we use a dynamic approach to determine the lower and upper positions of each line. Let's assume the candidate line CL_i is represented by a list of points. The procedure for defining the lower and upper positions (CL_{i-1} and CL_{i+1}) of this line is explained by Algorithm 3.

Algorithm 3 Defining the dynamic lower and upper position of each line.

Require: The candidate text-line i-1, i, i+1 are CL_{i-1}, CL_i, CL_{i+1}. List of points representing CL_i is $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$. The average height of the characters is $\overline{h_c}$.

if $ABS(CL_{i-1}, CL_i) \geq \overline{h_c}$ **then**
  $d_{lower} \leftarrow ABS(CL_{i-1}, CL_i)$
else
  $d_{lower} \leftarrow \overline{h_c}$
end if
if $ABS(CL_{i+1}, CL_i) \geq \overline{h_c}$ **then**
  $d_{upper} \leftarrow ABS(CL_{i+1}, CL_i)$
else
  $d_{upper} \leftarrow \overline{h_c}$
end if
$l_{cli} \leftarrow \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\} - d_{lower}$
$u_{cli} \leftarrow \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\} + d_{upper}$

The visual representation of the candidate text lines' positions is illustrated in Fig. 10. Let $E_{l_i:u_i,w}$ denote the energy map where w is the number of point representing the candidate

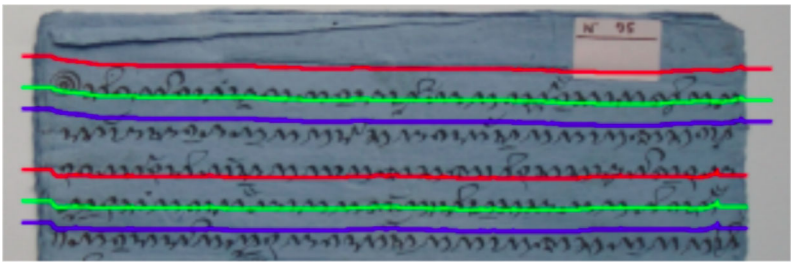


Fig. 10 The position of several candidate text lines (generated by Algo. 3). Each candidate line is marked in green, its lower boundary in red and its upper boundary in blue

line CL_i. The seam carving method consists of two steps: a forward step and a backward step.

In the forward step, we initialize seams at each vertical pixel from l_{cli} to u_{cli}. Each seam moves from the current column to the next column by computing the scores $M_{y,x}$ and $M_{y,x+1}$ as follow :

$$M_{y,x} = E_{y,x} \tag{1}$$

$$M_{y,x+1} = E_{y,x+1} + \min(M_{y-1:y+1,x-1}) \tag{2}$$

with $y \in [l_{cli}, u_{cli}]$ and $x \in [1, w]$. This process is sequentially done from the first point on the left to the last point in the list of points.

In the backward step, we construct the separating seam at the position with the lowest cumulative score of $M_{l_{cli}:u_{cli},w}$ then we go back from right to left by selecting the pixel with the lowest cost in the left column. To ensure a continuous separation, we only consider the adjacent pixels (those on the previous, current, and next lines).

3.2.2 Minimum spanning tree

Image binarization The binarization results play a crucial role in the Minimum Spanning Tree (MST) step. In the MST step, they help to construct the graph. For this purpose, we adapt the pre-trained model introduced in [44]. While we experimented with alternative thresholding methods such as simple thresholding or Otsu's thresholding, we found that the pre-trained model from [44] consistently yielded superior results.

Minimum spanning tree To refine and rectify the segmentation of text lines, we use Minimum Spanning Tree (MST) [46]. MST represents a subset of the edges from a connected, edge-weighted un-directed graph linking all vertices together, without forming any cycles and with the minimal total edge weight possible. The use of MST for adjusting segmentation results is detailed in Algorithm 4.

Algorithm 4 Use of MST for refinement segmentation results.

Require: The list of upper and lower polygons extracted from cast seams step.
1: Take the image inside the lower and upper polygon pair.
2: Get all connected components in the extracted image.
3: Build a graph from the defined representation points for each connected component.
4: Finding the optimal link from graph.
5: Extract polygons from graph.

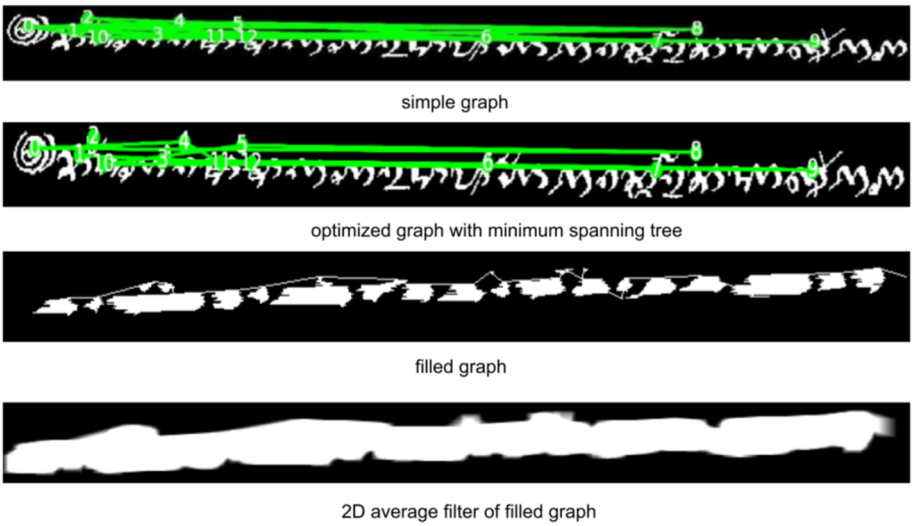


Fig. 11 Illustration of segmentation results in each step of MST. For easy observation, we just draw with few connected components in construction of graph

An illustration of the MST algorithm is provided in Fig. 11. This representation selectively visualizes a subset of connected components that correspond to text lines.

The MST algorithm connects all relevant points within these components in a manner that minimizes the total edge weight, resulting in a minimum spanning graph.

This process highlights the structural relationships between points within the text line components, facilitating the identification of coherent text areas.

After constructing the graph, a dilation operation is applied to enhance the delineation of these text areas. It is crucial to note that the effectiveness of the MST algorithm is significantly influenced by the quality of the input binarization. The accuracy of the binarization process directly impacts graph construction, as it determines which points are included in the connected components. Therefore, any errors in binarization may propagate through the graph construction process, adversely affecting the delineation of the resulting text areas.

4 Cham manuscripts image dataset

This section introduces the Cham manuscript dataset used in this work, comprising nearly 11,000 pages of Cham handwritten manuscripts written on "Chinese" paper. Figure 12 demonstrates some examples of Cham manuscripts in the dataset. The corpus encompasses manuscripts in two languages and scripts: Han Nom (about 5% of the corpus) and Middle Cham. To be more precise, we focused solely on the distribution of data related to the Cham language, excluding the Han Nom language from our analysis. A large part of the manuscripts can be dated (from Vietnamese seals) between 1702 and 1810, making this set of manuscripts a coherent corpus of 18th century texts. These manuscripts are the proof of Champa's possessions and their management. These documents relate the daily life of the inhabitants of ancient Panduranga during more than one century and complete our perception of the complex process, also little known, of the annexation of the Panduranga by the Nguyen dynasty. The alphabet used was originally borrowed from India to write inscriptions in San-



Fig. 12 Some examples of Cham manuscript images in our dataset

skrit. It was adapted to Cham pronunciation for the inscriptions written in ancient Cham. Manuscripts in Middle Cham continue this process of adapting an alphabet to a language that is not pronounced in the same way as the language for which it was created : the complexity of the Cham alphabet has adapted to the evolution of the langage, including Malay influences from the 16th century onwards or Vietnamese influences from the 18th. Thus, the Cham alphabet evolved further to adapt to changes in vocabulary and pronunciation. Each image includes one or two pages. For each page, the number of text-lines varies and can be up to 30. The distribution of text lines per image is illustrated in Fig. 13. Additional details regarding the linguistics or the collection process of the dataset can be found in [3].

We used a total of 627 manuscripts (approximately 8300 segmented text lines) for the evaluation. The dataset was divided into 125 manuscripts for training, 40 for validation (for fine-tuning docExtractor), and 462 for testing. Our linguistic experts were responsible for creating the ground truth data for the segmentation task. The primary objective is to ensure that all relevant text associated with the text lines is thoroughly covered. The boundaries established during this annotation process do not need to be excessively tight or loose; rather, they should be sufficient to encompass the entire text. This ground truth data is designed to effectively facilitate the text line transliteration process. Figure 14 illustrates some examples of ground truth for text-lines.

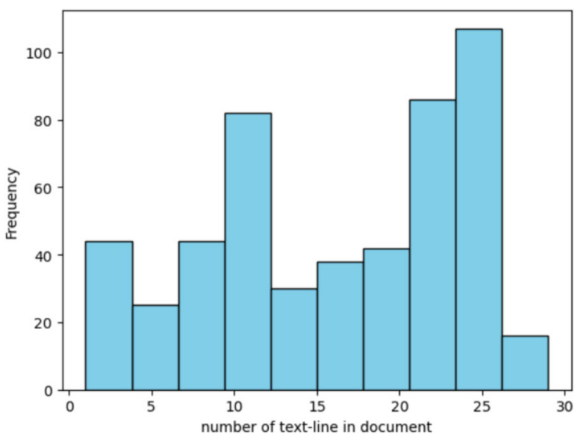


Fig. 13 Distribution of text lines per image

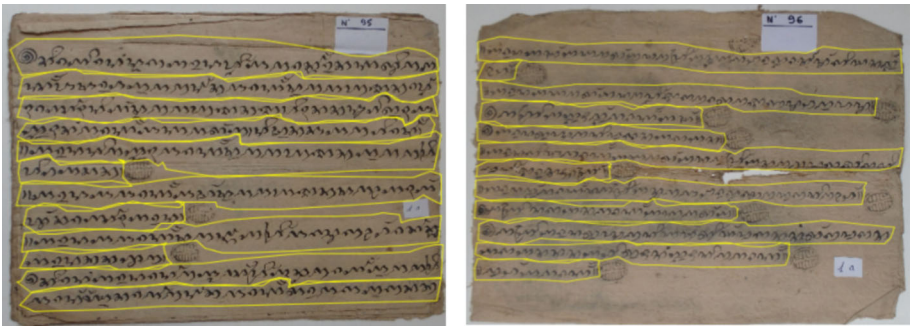


Fig. 14 Visualization of the ground truth for text line. Yellows boundary represent for text-lines

5 Experiments

5.1 Fine tuning docExtractor

docExtractor, initially introduced in [7], was originally trained to classify into four different categories (background, image, text, text border). However, for our task with Cham manuscripts, we only require two classes: background and text-line. Therefore, we modified the last fully connected layer from 4 to 2 outputs. Subsequently, the entire model was fine-tuned using 125 images of Cham manuscripts. We retained the same loss function that is CrossEntropy loss as in the original paper and selected the best model based on the performance on 40 validation images. The convergence of the fine-tuning process is illustrated in Fig. 15.

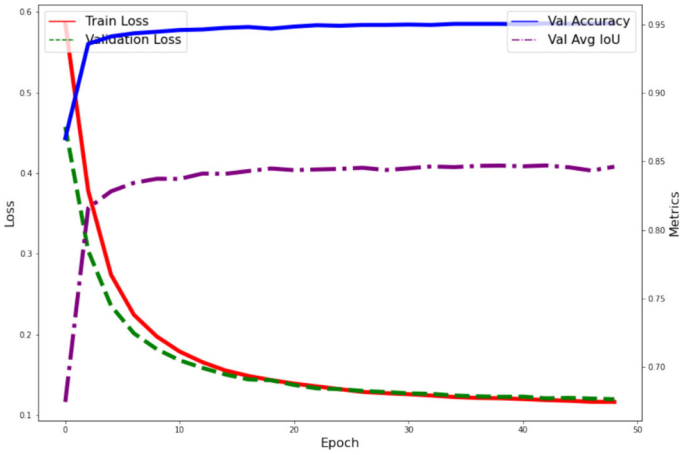


Fig. 15 The fine-tuning process of docExtractor on Cham manuscripts is illustrated, with training and validation loss depicted on the left y-axis. Concurrently, the right y-axis displays the classification category accuracy (represented in blue) and the Intersection over Union (IoU) score (shown in purple) evaluated on the validation set

Table 1 Hyper parameters used in our experiments

Name of parameters	Value
Algorithm 1	
Threshold for height distance T_{cch}	5
Threshold for angle distance T_{cca}	20
Threshold for error T_{err}	5.1
Algorithm 2	
Threshold for height T_{height}	46
Threshold for error T_{err}	5
Algorithm 3	
The threshold for average height of the characters $\overline{h_c}$	35
Algorithm 4	
Kernel size of 2D averaging filter	15

5.2 Hyperparameters

So far, the hyperparameters (splitting line and merging line) as well as the kernel size of the 2D averaging filter were chosen mainly by analyzing the characteristics of character in the documents such as height and line distance. The hyperparameters settings are summarized in Table 1. Moreover, the connected components from the baseline extraction image of less than 100 pixels are removed. For image binarization, we use the trained model SAE from [44] whose weights are obtained from the H-DIBCO 2016 dataset [47].

5.3 Evaluation metrics

On the one hand, since we do not have a clearly defined ground truth format (with close or approximate boundaries), it may not be fair to directly compare the performance of the different methods. On the other hand, the result of this step will be used for the transliteration phase, which makes it more relevant to use transliteration metrics to evaluate each method. Based on this observation, we use a combination of segmentation and transliteration measures. For segmentation evaluation, we use pixel- and object-level metrics. At the pixel level, evaluation metrics include intersection over union (IoU), Precision, Recall, and F1 score, defined as follows :

- Intersection-over-Union (IoU):

$$IoU = \frac{TP}{TP + FP + FN} \tag{3}$$

- Precision (P):

$$Precision = \frac{TP}{TP + FP} \tag{4}$$

- Recall (R):

$$Recall = \frac{TP}{TP + FN} \tag{5}$$

- F1-score (F1):

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{6}$$

At the object level, Average Precision (AP) is employed to assess the performance of object detection or segmentation algorithms. AP can be calculated at different IoU thresholds (e.g., 50%, 75%), where IoU measures the overlap between the predicted segmentation and the ground truth segmentation at those thresholds.

$$AP@[.5],\ AP@[.75] = \text{average precision at } 50\%, 75\% \text{ IoU} \tag{7}$$

In this work, rather than directly evaluating the performance of text line segmentation, we focus on assessing the impact of text line segmentation results on the text transliteration task. To accomplish this, we utilize the state-of-the-art Two Step Sequence Transformer (TSST) model [48]. The model used in this study is based on the transformer and sequence-to-sequence architecture and was previously trained according to the methods described in [48]. For the transliteration aspect, we use Word Error Rate (WER) and Character Error Rate (CER) as evaluation metrics, following the methodology outlined in [48]. CER assesses model performance by computing the Levenshtein distance between the ground truth and the predicted output at the character level. Similarly, WER calculates the Levenshtein distance, but at the word level instead of character-by-character as in CER. These metrics are computed as follows:

$$CER = \frac{S_c + D_c + I_c}{S_c + D_c + C_c} \quad WER = \frac{S_w + D_w + I_w}{S_w + D_w + C_w}$$

where:

- S_c, S_w: the number of substituted **characters/ words**.
- D_c, D_w: the number of deleted **characters/ words**.
- I_c, I_w: the number of inserted **characters/ words**.
- C_c, C_w: the number of correct **characters/ words**.

5.4 Experimental results

5.4.1 Ablation study

In this section, we conduct an ablation study using various configurations of the main steps in the proposed pipeline for the Cham manuscripts dataset. This exploration aims to understand how each step affects the final results.

Post processing in baseline extraction step we study the importance of using the splitting and merging algorithm in baseline extraction step. Results of ablation study are shown in Table 2.

We evaluate three scenarios: without post processing, only merging and both splitting and merging on two sets: the entire dataset and the subset of manuscripts that require post-processing. Generally, we observe a slight improvement when post-processing is applied, with even lower results when both splitting and merging are utilized. This result can be attributed to the presence of noise in certain documents, where merging or splitting may not fully correct the segmentation.

For the subset, we achieve better scores with using both merging and splitting techniques, particularly in terms of precision, although recall and IoU scores are lower. This discrepancy can be attributed to the fact that while the proposed splitting algorithm performs well in successful cases (as shown in Fig. 16), it encounters challenges with unsuccessful cases, often involving overlapping text lines. The lack of information regarding the thickness of

Table 2 Results with and without using the post-processing algorithm in baseline extraction step

Experiments	Pixel Metrics				Object Metrics	
	IoU	Precision	Recall	F1	AP@[.5]	AP@[.75]
Full dataset						
Without post processing	0.426	**0.977**	0.310	0.590	0.151	0
Only Merging	**0.428**	0.974	**0.434**	**0.593**	**0.153**	0
Splitting and Merging	0.419	0.975	0.425	0.586	0.140	0
Subset*						
Without post processing	0.412	0.977	0.414	0.573	0.157	0
Only Merging	**0.422**	0.968	**0.426**	**0.585**	**0.168**	0
Splitting and Merging	0.420	**0.984**	0.422	**0.585**	0.132	0

Best results for each set are in bold
*Evaluation on 97 manuscripts images needed to post-processing text-lines

these touching lines makes accurate splitting difficult, even when the splitting conditions are met (as illustrated in Fig. 17). Additionally, during the line-splitting process, main text lines may also be inadvertently cut, resulting in reduced precision scores. Consistently low AP@[.5] and AP@[.75] scores indicate that the segmentation results do not closely match the ground truth as expected because they are not accurate. The use of seam carving and MST is necessary to further improve results.

Image binarization Table 3 shows the results of the ablation study for image binarization. In this comparison, we used a basic energy map and maintained a consistent MST algorithm (kernel size 17x17). The highest results were obtained with the SAE pre-trained model [44] with 0.647, 0.893, 0.726, 0.785 for IoU, Precision, Recall, and F1 metrics. Although applying the pre-trained model [44] does not give optimal results when binarizing the entire collection, its performance still outperforms simple thresholding and Otsu binarization methods. We believe that fine-tuning this pre-trained model could further improve the overall accuracy.

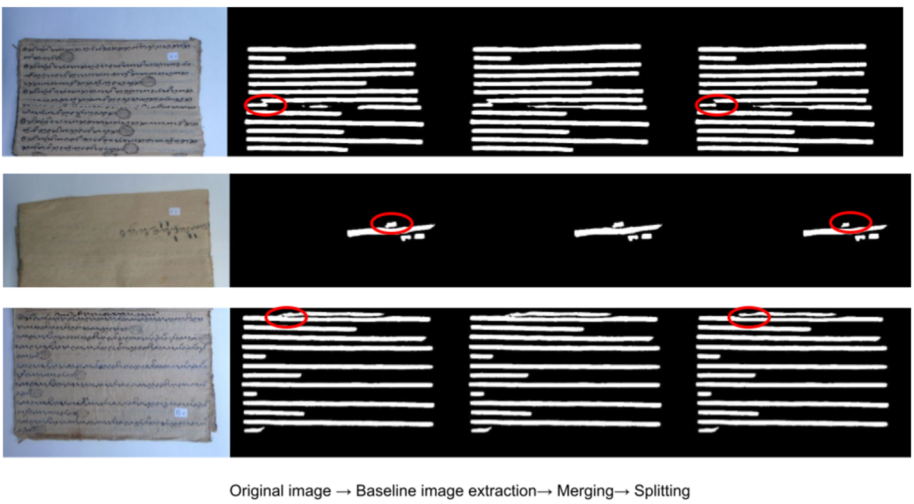


Fig. 16 Illustration of successful text-line splitting

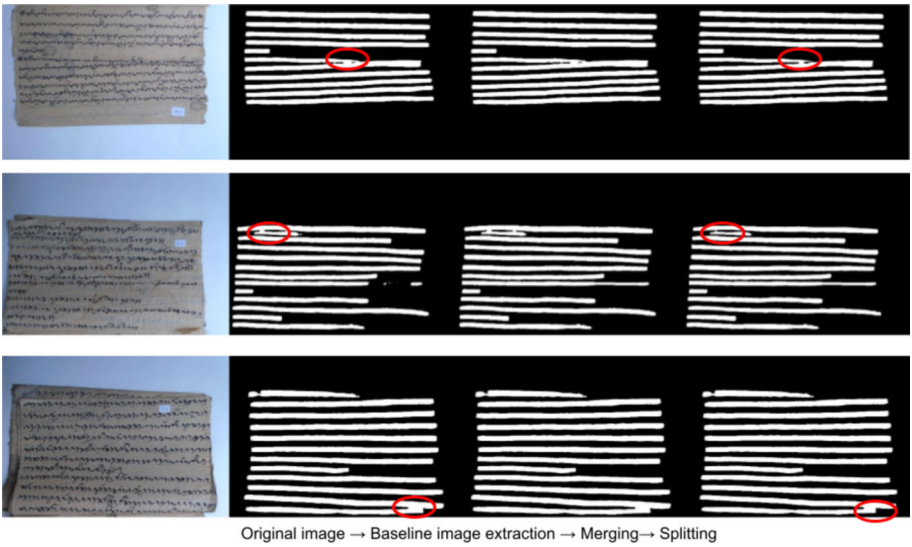


Fig. 17 Illustration of unsuccessful splitting in case of touching text lines

Table 3 Text line segmentation results using different binarization models

Experiments	Pixel Metrics				Object Metrics	
	IoU	Precision	Recall	F1	AP@[.5]	AP@[.75]
Otsu's Thresholding [49]	0.453	0.569	0.545	0.536	0.580	0.247
Simple Thresholding	0.633	**0.903**	0.676	0.755	0.823	0.213
Pre-trained SAE [44]	**0.647**	0.893	**0.726**	**0.785**	**0.867**	**0.453**

Table 4 Influence of kernel size of Minimum Spanning Tree

Experiments	Pixel Metrics				Object Metrics	
Kernel size (wxh)	IoU	Precision	Recall	F1	AP@[.5]	AP@[.75]
w/o MST	0.483	0.899	0.500	0.614	0.674	0
7x7	0.477	**0.937**	0.487	0.629	0.569	0
9x9	0.528	0.932	0.542	0.673	0.718	0
11x11	0.573	0.924	0.593	0.709	0.781	0.05
13x13	0.613	0.916	0.641	0.739	0.819	0.157
15x15	0.646	0.905	0.685	0.765	0.844	0.316
17x17	**0.674**	0.893	**0.726**	**0.785**	**0.867**	**0.453**

Table 5 Comparison with others text line segmentation methods and off-the-shelf software on Cham manuscripts

Experiments	Pixel Metrics				Object Metrics	
	IoU	Precision	Recall	F1	AP@[.5]	AP@[.75]
doc-UFCN [6]	0.211	0.619	0.242	0.325	00.06	0
docExtractor [7]	0.426	0.977	0.431	0.590	0.151	0
Transkibus [8]	0.295	0.930	0.299	0.444	0.05	0
eScriptorium [9]	0.531	0.882	0.565	0.648	0.649	0.192
LayoutParser [40]	0.626	0.722	**0.837**	0.751	**0.849**	0.230
Ours*	**0.646**	**0.905**	0.685	**0.765**	0.844	**0.316**

* Our configuration uses merging post-processing in conjunction with a pre-trained SAE model and MST with a kernel size of 15x15.

Minimum spanning tree We performed an evaluation of the MST technique using different kernel sizes (Table 4). Our results reveal that using MST leads to improved accuracy compared to not using MST, especially with smaller kernels. Interestingly, as the kernel size increases, the precision decreases slightly, but a notable improvement in the recall score is observed. This phenomenon can be attributed to the nature of the ground truth data.

Since ground truth annotations are not strictly closed, larger kernel sizes tend to perform better. However, we only observed marginal changes in the transliteration of the segmentation results from these models. The slight decrease in transliteration accuracy could be due to the inclusion of adjacent text from neighboring lines when the lines of text are spaced further apart, thus affecting the transliteration process.

5.4.2 Comparison with state-of-the-art methods

In Table 5, we present the performance of different methods, off-the-shelf software and available pre-trained models on the Cham manuscripts dataset. For the doc-UFCN, docExtractor, and Layout Parser, we used fine-tuned pretrained models on our Cham manuscript dataset. doc-UFCN's weights is taken from their public implementation with a mixed dataset from Balsac, Horae, READ-Simple, READ-complex, DIVA-HisDB.[2] For Transkibus and eScriptorium, we used the models from the official websites[3] and.[4]

We observe that docExtractor and Transkribus exhibit superior performance in Precision scores, achieving approximately 97% and 93%, respectively. This can be attributed to their focus on baseline extraction, which facilitates accurate segmentation of the text line area, known as the simpler part of text lines. However, both models demonstrate relatively low recall metrics and IoU scores, indicating a significant number of missed descenders, ascenders, and diacritics. For the Layout Parser model, it is evident that while it achieves a notably high recall, it exhibits relatively low precision. This discrepancy can be attributed to the use of loose bounding boxes. The high recall indicates the model's effectiveness in identifying a significant number of relevant areas within the layout; however, the low precision suggests that many of these identified elements may not be accurately localized, resulting in a higher rate of false positives. Thus, the performance characteristics of the Layout Parser model highlight

[2] https://pypi.org/project/doc-ufcn/

[3] https://readcoop.eu/transkribus/

[4] https://www.sofer.info/

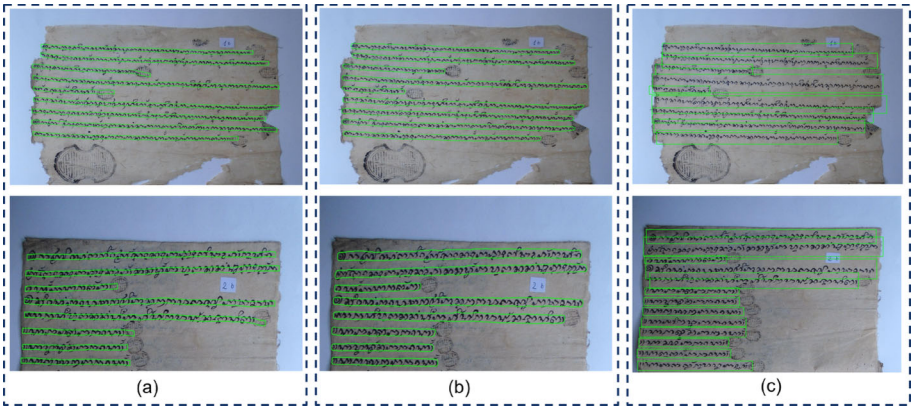


Fig. 18 Segmentation results obtained by (a) Transkibus; (b) docExtractor and (c) Layout Parser

the inherent trade-off between recall and precision, which is influenced by the bounding box strategy employed in its architecture. Examples of results obtained with docExtractor, Transkribus, LayoutParser are illustrated in Fig. 18. Our model, eScriptorum, LayoutParser show considerable improvement in IoU and Recall scores, leading to an enhanced F1 score as they attempt to segment additional components. However, this sometimes results in inaccuracies, leading to a slight decrease in precision compared to models from docExtractor and Transkribus. Nonetheless, our proposed model manages to maintain precision levels close to the top-performing models while significantly improving on other metrics. This improvement is evident in Fig. 19, where the proposed method accurately segments entire text-lines and diacritics, while eScriptorium produces incomplete text-lines and misses some short



Fig. 19 Qualitative comparison between the proposed approach and eScriptorium. Green polygons represent the results of the proposed approach, while red polygons represent eScriptorium's results

Table 6 Comparison of off-the-shelf software performance on Cham manuscripts based on a set of transliteration metrics

Model	WER	CER
doc-UFCN [6]	77.06	64.74
Transkribus [8]	61.19	41.89
docExtractor [7]	51.85	30.54
LayoutParser [40]	42.39	24.64
eScriptorium [9]	42.08	26.23
Ours without MST	40.63	23.52
Ours with MST (kernel size 17)	38.49	22.0
Ours with MST (kernel size 15)	38.16	21.73
Manually segmentation	**23.42**	**10.82**

text-lines, resulting in a 10% improvement for each metric compared to eScriptorium. In comparison with LayoutParser, our method shows a 2% improvement in IoU and 1% in F1 score.

5.4.3 Evaluating the impact of text line segmentation quality on transliteration task

As the output from the text line segmentation step serves as input for transliteration, it is essential to evaluate the performance of these results using transliteration metrics. In all experiments, we employed the same pre-trained TSST model for Cham language [48] with text line segmentation results of different models (Table 6).

We observe relatively high prediction errors in baseline models such as doc-UFCN [6] and Transkribus [8], mainly due to inaccurate baseline extraction. Notably, there is a significant improvement in models like docExtractor [7], eScriptorum [9] and LayoutParser [40] when baseline accuracy is enhanced. Our model achieved superior results, with approximately 38.16% and 21.73% WER and CER respectively, indicating an improvement of around 4% for each metric compared to eScriptorirum and LayoutParser. However, even with these improvements, the results are still notably far from manual segmentation, exhibiting approximately 15% higher WER and 11% higher CER. This discrepancy underscores the importance of precise segmentation to ensure reliable transliteration systems.

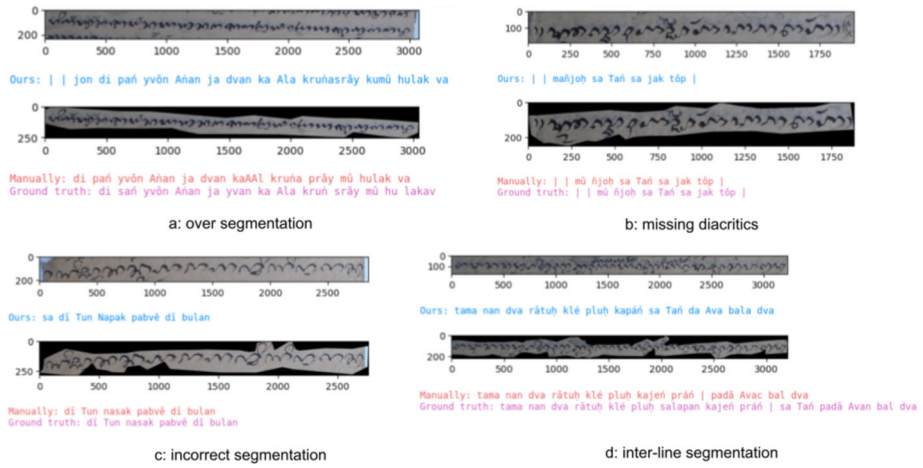


Fig. 20 Errors in transliteration occur due to inaccurate text line segmentation

Notably, small segmentation errors can lead to incorrect character and word predictions. Some examples are illustrated in Fig. 20. We can observe that over-segmentation, missing diacritics, incorrect segmentation, and inter-line segmentation are reasons for incorrect transliteration.

6 Conclusions and future works

In this paper, we propose an efficient method to address the challenge of segmenting text lines in Cham manuscripts, leveraging both pre-trained deep learning models and traditional image processing techniques. Through comprehensive experiments conducted on our Cham manuscript dataset and analysis of various metrics encompassing both segmentation and transliteration, we demonstrate the effectiveness of our approach compared with the state-of-the-art methods. Experimental results also confirm the role of the text line segmentation step in the whole process of document analysis. In the future, we aim to enhance the splitting algorithm to facilitate text line segmentation without sacrificing crucial information. Additionally, given the disparity between manual and model-based segmentation results, fine-tuning the transliteration model becomes imperative, especially considering that the current model is trained solely on manual segmentation. Finally, our objective is to establish a comprehensive end-to-end framework for the comprehensive understanding of Cham manuscripts.

Funding This work is supported by the French Research Agency (ANR) in the framework of the ChAMDOC Project, n°ANR-19-CE27-0018-02.

Data Availability Data will be made available on reasonable request.

Declarations

Conflicts of Interest We have no conflict of interest to declare.

References

1. Kesiman MWA (2018) Document image analysis of Balinese palm leaf manuscripts: Analyse d'images de documents des manuscrits balinais sur feuilles de palmier. PhD thesis, La Rochelle
2. Valy D (2020) Document image analysis and text recognition on khmer historical manuscripts. PhD thesis, Catholic University of Louvain, Louvain-la-Neuve, Belgium
3. Nguyen TN (2023) Segmentation, recognition and indexing of cham characters in cham documents. PhD thesis, Université de La Rochelle
4. O'Gorman L (1993) The document spectrum for page layout analysis. IEEE Trans Pattern Anal Mach Intell 15(11):1162–1173
5. Nguyen T-N, Burie J-C, Le T-L, Schweyer A-V (2022) An effective method for text line segmentation in historical document images. In: 2022 26th International conference on pattern recognition (ICPR). IEEE, pp 1593–1599
6. Boillet M, Kermorvant C, Paquet T (2021) Multiple document datasets pre-training improves text line detection with deep neural networks. In: 2020 25th International conference on pattern recognition (ICPR). IEEE, pp 2134–2141
7. Monnier T, Aubry M (2020) docextractor: An off-the-shelf historical document element extraction. In: 2020 17th International conference on frontiers in handwriting recognition (ICFHR). IEEE, pp 91–96
8. Kahle P, Colutto S, Hackl G, Mühlberger G (2017) Transkribus-a service platform for transcription, recognition and retrieval of historical documents. In: 2017 14th IAPR International conference on document analysis and recognition (ICDAR). IEEE, vol 4, pp 19–24

9. Kiessling B, Tissot R, Stokes P, Ezra DSB (2019) escriptorium: an open source platform for historical document analysis. In: 2019 International conference on document analysis and recognition workshops (ICDARW). IEEE, vol 2, pp 19–19
10. Likforman-Sulem L, Zahour A, Taconet B (2007) Text line segmentation of historical documents: a survey. Int J Document Anal Recognit (IJDAR) 9:123–138
11. Manmatha R, Srimal N (1999) Scale space technique for word segmentation in handwritten documents. In: Proceedings of the second international conference on scale-space theories in computer vision. SCALE-SPACE '99, Springer, Berlin, Heidelberg, pp 22–33
12. Manmatha R, Srimal N (2002) Scale space technique for word segmentation in handwritten documents. In: Scale-Space Theories in Computer Vision: Second International Conference, Scale-Space'99 Corfu, Greece, September 26–27, 1999 Proceedings. Springer, pp 22–33
13. Valy D, Verleysen M, Sok K (2017) Line segmentation for grayscale text images of khmer palm leaf manuscripts. In: 2017 Seventh International conference on image processing theory, tools and applications (IPTA). IEEE, pp 1–6
14. Arvanitopoulos N, Süsstrunk S (2014) Seam carving for text line extraction on color and grayscale historical manuscripts. In: 2014 14th International conference on frontiers in handwriting recognition, pp 726–731. https://doi.org/10.1109/ICFHR.2014.127
15. Boulid Y, Souhar A, Elkettani MY (2015) Arabic handwritten text line extraction using connected component analysis from a multi agent perspective. In: 2015 15th International conference on intelligent systems design and applications (ISDA), pp 80–87. https://doi.org/10.1109/ISDA.2015.7489204
16. Chamchong R, Fung CC (2012) Text line extraction using adaptive partial projection for palm leaf manuscripts from thailand. In: 2012 International conference on frontiers in handwriting recognition. IEEE, pp 588–593
17. Valy D, Verleysen M, Sok K (2016) Line segmentation approach for ancient palm leaf manuscripts using competitive learning algorithm. In: 2016 15th International conference on frontiers in handwriting recognition (ICFHR), pp 108–113. https://doi.org/10.1109/ICFHR.2016.0032
18. Sanasam I, Choudhary P, Singh KM (2020) Line and word segmentation of handwritten text document by mid-point detection and gap trailing. Multimed Tools Appl 79(41):30135–30150
19. Asi A, Saabni R, El-Sana J (2011) Text line segmentation for gray scale historical document images. In: Proceedings of the 2011 workshop on historical document imaging and processing. HIP '11, pp 120–126. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/2037342.2037362
20. Seuret M, Ben Ezra DS, Liwicki M (2017) Robust heartbeat-based line segmentation methods for regular texts and paratextual elements. In: Proceedings of the 4th International workshop on historical document imaging and processing. HIP2017, pp 71–76. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3151509.3151521
21. Alberti M, Vögtlin L, Pondenkandath V, Seuret M, Ingold R, Liwicki M (2019) Labeling, cutting, grouping: an efficient text line segmentation method for medieval manuscripts. CoRR. abs/1906.11894
22. Surinta O, Holtkamp M, Karabaa F, Van Oosten J-P, Schomaker L, Wiering M (2014) A path planning for line segmentation of handwritten documents. In: 2014 14th International conference on frontiers in handwriting recognition, pp 175–180. https://doi.org/10.1109/ICFHR.2014.37
23. Nicolaou A, Gatos B (2009) Handwritten text line segmentation by shredding text into its lines. In: 2009 10th International conference on document analysis and recognition, pp 626–630. https://doi.org/10.1109/ICDAR.2009.243
24. Cohen R, Dinstein I, El-Sana J, Kedem K (2014) Using scale-space anisotropic smoothing for text line extraction in historical documents, pp. 349–358. https://doi.org/10.1007/978-3-319-11758-4_38
25. Zahour A, Taconet B, Likforman-Sulem L, Boussellaa W (2009) Overlapping and multi-touching text-line segmentation by block covering analysis. Pattern Anal Appl 12:335–351
26. Fateh A, Rezvani M, Tajary A, Fateh M (2023) Persian printed text line detection based on font size. Multimed Tools Appl 82(2):2393–2418
27. Wong KY, Casey RG, Wahl FM (1982) Document analysis system. IBM J Res Dev 26(6):647–656
28. Kesiman MWA, Burie J-C, Ogier J-M (2016) A new scheme for text line and character segmentation from gray scale images of palm leaf manuscript. In: 2016 15th International conference on frontiers in handwriting recognition (ICFHR). IEEE, pp 325–330
29. Shi Z, Setlur S, Govindaraju V (2009) A steerable directional local profile technique for extraction of handwritten arabic text lines. In: 2009 10th International conference on document analysis and recognition. IEEE, pp 176–180
30. Shi Z, Setlur S, Govindaraju V (2005) Text extraction from gray scale historical document images using adaptive local connectivity map. In: Eighth International conference on document analysis and recognition (ICDAR'05). IEEE, pp 794–798

31. Barakat, B.K., Droby, A., Kassis, M., El-Sana, J (2021) Text line segmentation for challenging handwritten document images using fully convolutional network. CoRR. abs/2101.08299. 2101.08299
32. Barakat BK, Droby A, Alasam R, Madi B, Rabaev I, Shammes R, El-Sana J (2020) Unsupervised text line segmentation. CoRR. abs/2003.08632. 2003.08632
33. Grüning T, Leifert G, Strauß T, Michael J, Labahn R (2019) A two-stage method for text line detection in historical documents. Int J Document Anal Recognit (IJDAR) 22(3):285–302
34. Mechi O, Mehri M, Ingold R, Essoukri Ben Amara N (2019) Text line segmentation in historical document images using an adaptive u-net architecture. In: 2019 International conference on document analysis and recognition (ICDAR), pp 369–374. https://doi.org/10.1109/ICDAR.2019.00066
35. Vo QN, Kim SH, Yang HJ, Lee GS (2018) Text line segmentation using a fully convolutional network in handwritten document images. IET Image Process 12(3):438–446
36. Renton G, Chatelain C, Adam S, Kermorvant C, Paquet T (2017) Handwritten text line segmentation using fully convolutional network. In: 2017 14th IAPR International conference on document analysis and recognition (ICDAR). IEEE, vol 5, pp 5–9
37. Jindal A, Ghosh R (2023) Text line segmentation in indian ancient handwritten documents using faster r-cnn. Multimed Tools Appl 82(7):10703–10722
38. Long J, Shelhamer E, Darrell T (2015) Fully convolutional networks for semantic segmentation. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 3431–3440
39. Ronneberger O, Fischer P, Brox T (2015) U-net: Convolutional networks for biomedical image segmentation. In: Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18. Springer, pp 234–241
40. Shen Z, Zhang R, Dell M, Lee BCG, Carlson J, Li W (2021) Layoutparser: A unified toolkit for deep learning based document image analysis. In: Document analysis and recognition–ICDAR 2021: 16th International Conference, Lausanne, Switzerland, September 5–10, 2021, Proceedings, Part I 16. Springer, pp 131–146
41. Gruuening T, Leifert G, Strauss T, Labahn R (2017) A robust and binarization-free approach for text line detection in historical documents. In: 2017 14th IAPR International conference on document analysis and recognition (ICDAR). IEEE, vol 1, pp 236–241
42. Grüning T, Labahn R, Diem M, Kleber F, Fiel S (2018) Read-bad: a new dataset and evaluation scheme for baseline detection in archival documents. In: 2018 13th IAPR International workshop on document analysis systems (DAS). IEEE, pp 351–356
43. Simistira F, Bouillon M, Seuret M, Würsch M, Alberti M, Ingold R, Liwicki M (2017) Icdar2017 competition on layout analysis for challenging medieval manuscripts. In: 2017 14th IAPR International conference on document analysis and recognition (ICDAR), vol 01, pp 1361–1370. https://doi.org/10.1109/ICDAR.2017.223
44. Calvo-Zaragoza J, Gallego A-J (2019) A selectional auto-encoder approach for document image binarization. Pattern Recognit 86:37–47. https://doi.org/10.1016/j.patcog.2018.08.011
45. Levi G, Montanari U (1970) A grey-weighted skeleton. Inf Control 17(1):62–91. https://doi.org/10.1016/S0019-9958(70)80006-7
46. Kruskal JB (1956) On the shortest spanning subtree of a graph and the traveling salesman problem. Proc Am Math Soc 7(1):48–50
47. Pratikakis I, Zagoris K, Barlas G, Gatos B (2016) Icfhr2016 handwritten document image binarization contest (h-dibco 2016). In: 2016 15th International conference on frontiers in handwriting recognition (ICFHR). IEEE, pp 619–623
48. Nguyen T-N, Burie J-C, Le T-L, Schweyer A-V (2023) Two-step sequence transformer based method for cham to latin script transliteration. In: Proceedings of the 7th international workshop on historical document imaging and processing, pp 25–30
49. Otsu N et al (1975) A threshold selection method from gray-level histograms. Automatica 11(285–296):23–27

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.